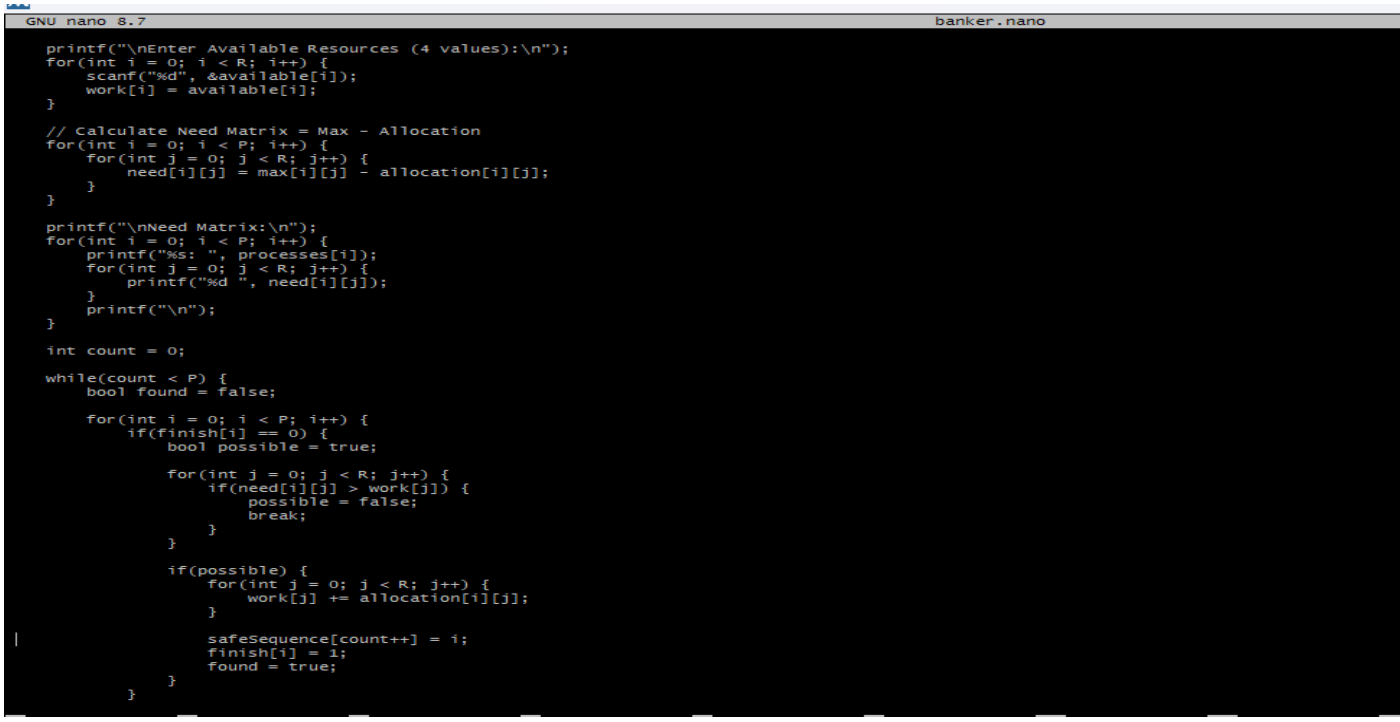
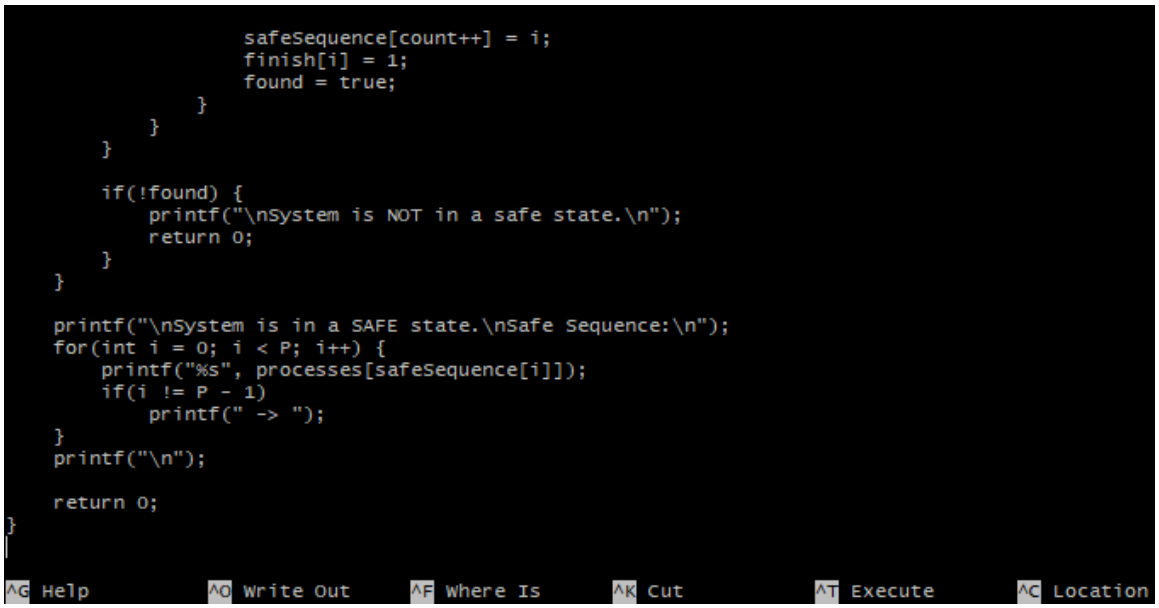
Practical 7

Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

```
  GNU nano 8.7                                                    banker.nano
#include <stdio.h>
#include <stdbool.h>

#define P 5   // Number of Processes
#define R 4   // Number of Resource Types

int main() {
    int allocation[P][R], max[P][R], need[P][R];
    int available[R];
    int finish[P] = {0};
    int safeSequence[P];
    int work[R];

    char *processes[P] = {
        "Google Drive",
        "Firefox",
        "Word Processor",
        "Excel",
        "PowerPoint"
    };

    char *resources[R] = {
        "Printer",
        "ROM",
        "Hard Disk",
        "RAM"
    };

    printf("Enter Allocation Matrix (5x4):\n");
    for(int i = 0; i < P; i++) {
        for(int j = 0; j < R; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Maximum Matrix (5x4):\n");
    for(int i = 0; i < P; i++) {
        for(int j = 0; j < R; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources (4 values):\n");
    for(int i = 0; i < R; i++) {
        scanf("%d", &available[i]);
```


GNU nano 8.7
banker.nano
printf("\nEnter Available Resources (4 values):\n");
for(int i = 0; i < R; i++) {
scanf("%d", &available[i]);
work[i] = available[i];
}
// Calculate Need Matrix = Max - Allocation
for(int i = 0; i < P; i++) {
for(int j = 0; j < R; j++) {
need[i][j] = max[i][j] - allocation[i][j];
}
}
printf("\nNeed Matrix:\n");
for(int i = 0; i < P; i++) {
printf("%s: ", processes[i]);
for(int j = 0; j < R; j++) {
printf("%d ", need[i][j]);
}
printf("\n");
}
int count = 0;
while(count < P) {
bool found = false;
for(int i = 0; i < P; i++) {
if(finish[i] == 0) {
bool possible = true;
for(int j = 0; j < R; j++) {
if(need[i][j] > work[j]) {
possible = false;
break;
}
}
if(possible) {
for(int j = 0; j < R; j++) {
work[j] += allocation[i][j];
}
safeSequence[count++] = i;
finish[i] = 1;
found = true;
}
}


safeSequence[count++] = i;
finish[i] = 1;
found = true;
}
}
}
if(!found) {
printf("\nSystem is NOT in a safe state.\n");
return 0;
}
}
printf("\nSystem is in a SAFE state.\nSafe Sequence:\n");
for(int i = 0; i < P; i++) {
printf("%s", processes[safeSequence[i]]);
if(i != P - 1)
printf(" -> ");
}
printf("\n");
return 0;
}
^G Help
^O Write Out
^F Where Is
^K Cut
^T Execute
^C Location

```
HP@DESKTOP-B4BD73N MINGW64 ~
$ nano banker.c

HP@DESKTOP-B4BD73N MINGW64 ~
$ gcc banker.c -o banker

HP@DESKTOP-B4BD73N MINGW64 ~
$ ./banker
Enter Allocation Matrix (5x4):
0 1 0 3
2 0 0 1
3 0 2 1
2 1 1 0
0 0 2 2

Enter Maximum Matrix (5x4):
7 5 3 4
3 2 2 2
9 0 2 2
2 2 2 2
4 3 3 3

Enter Available Resources (4 values):
3 3 2 2

Need Matrix:
Google Drive: 7 4 3 1
Firefox: 1 2 2 1
Word Processor: 6 0 0 1
Excel: 0 1 1 2
PowerPoint: 4 3 1 1

System is in a SAFE state.
Safe Sequence:
Firefox -> Excel -> PowerPoint -> Google Drive -> Word Processor

HP@DESKTOP-B4BD73N MINGW64 ~
$
```